

Netto+ App Reverse Engineering Findings

Overview

Target: Netto+ Android app (dk.dsg.netto)
APK source: Extracted from GrapheneOS device via ADB
Framework: Kotlin, standard Android (DEX bytecode)
Goal: Intercept basket/scan events for expense tracking

Tools Used

Tool	Purpose
ADB	APK extraction, device communication
JADX-GUI	Static analysis / decompilation
Detect-It-Easy	Framework identification
apktool 3.0.1	APK decompile/recompile
uber-apk-signer	APK re-signing
Frida 17.8.3	Dynamic instrumentation
Android Emulator (API 34, x86_64)	Root environment for Frida
Postman	API endpoint testing

Hardcoded Secrets Found

SGP Payment API

```
Key:   SGP_API_URL
Value: https://p-sgpayment.dsgapps.dk/api

Key:   SGP_SECRET
Value:
VlR7kvrn1vlWnVKpbuSwfLY8j65Yh6reFbj7yKAdIkJUgsvRQSZEEv3aYghr0WoJLsjZcwqCycIDB
ELgLIJgdwfyDpslWuINDku410q62p2uigDaS9t8W4fyUpVLWetI552g7sZwhRpvL76rWpSkzA760p0
```

UMabeJzP956PkoIBUZKlcKfmKMB2DL5pDFBSA5WNPao0D0jBYq8HRUklP4fUbXusahm8SXxYwv3s70
EGMx01qxnksi8pyYfGhN3s

Viking Basket API

Key: VIKING_API_URL
Value: https://p-heimdalbackend.dsgapps.dk/api

Key: HEIMDAL_NATIVE_APP_TOKEN_SECRET
Value: Dimh6bKKMsboxdGgkM0rxoUGgNgFKlYIIbUhkEsms

Key: SHOPGUN_STORE_ID
Value: 9ba51

Membership API

Key: MEMBERSHIP_SECRET
Value: 038ac710-8fbd-4da1-96b3-43fda8f5c81c

Key: MEMBERSHIP_ISSUER
Value: a481276d-dd24-4d3c-897e-ff2115818a69

Key: MEMBERSHIP_BARCODE_API
Value: https://api.sallinggroup.com/

Tenant Info

Key: TENANT_ID (DK)
Value: 4

Key: TENANT_ALIAS (DK)
Value: TID-2Y7JRG

Key: TENANT_ALIAS_HEADER_NAME
Value: X-tenantAlias

Key: TENANT_ID (DE)
Value: 9

Key: TENANT_ALIAS (DE)
Value: TID-KR2RT5

Note: The German tenant (TID-KR2RT5 , tenantId=9) confirms DSG runs the same backend stack for Netto Germany — potential future expansion target.

Backend Architecture

The app communicates with four backends:

1. IDP (idp.dsgapps.dk / customervalidate.sallinggroup.com)

Identity provider — handles all authentication, token exchange and refresh.

2. Heimdal Backend (p-heimdalbackend.dsgapps.dk)

Core basket/scan-and-go API. Named internally as "Viking Basket".

3. Club API (p-club.dsgapps.dk)

Loyalty/membership API. Handles receipt history, settings and member offers. Most useful backend for expense tracking — returns full historical receipt data for all purchases regardless of payment method.

4. SGP Payment (p-sgpayment.dsgapps.dk)

Handles payment processing via Netaxept. Cert pinning only applies to *.meewallet.com (payment terminal SDK), not to any of the above backends.

Authentication

Flow: Authorization Code + PKCE (no client secret — public mobile client)

Identity Provider: Salling Group (customervalidate.sallinggroup.com / idp.dsgapps.dk)

Underlying identity platform: Gigya (SAP Customer Data Cloud)

Gigya credentials

```
API key: 3_8MtnEg0K6I9nk0rVGsFLTmrkAJhJAm7b_lRQH06071xnIM2MiIQ8aNoa-FOFY6wA
Base URL: https://accounts.eu1.gigya.com
```

Full auth flow

Step 1 — Gigya login

```
POST https://accounts.eu1.gigya.com/accounts.login
Content-Type: application/x-www-form-urlencoded
```

```
apiKey={gigya_api_key}
&loginID={email}
&password={password}
&format=json
```

Returns `sessionInfo.cookieValue` (the Gigya session token).

Step 2 — Get Gigya JWT

```
POST https://accounts.eu1.gigya.com/accounts.getJWT
Content-Type: application/x-www-form-urlencoded
```

```
apiKey={gigya_api_key}
&login_token={gigya_session_token}
&fields=data,profile
&format=json
```

Returns `id_token` (Gigya JWT) — this is passed as `login_hint` to the DSG IDP.

Step 3 — Authorization (Chrome Custom Tab on Android)

The app opens this URL in a Chrome Custom Tab. The IDP uses the `login_hint` (Gigya JWT) to auto-authenticate the user via the embedded Gigya widget, then redirects back to the app with an auth code.

```
GET https://customervalidate.sallinggroup.com/apps
?clientId=scan-and-go-native
&tenantId=4
&channel=ScanAndGo
&clientFlow=gigyaWithNemIdNew
&code_challenge_method=S256
&code_challenge={pkce_challenge}
&nonce={random_6}
&clientTraceId={install_id}
&emailOrPhone={email}
&login_hint={gigya_jwt}
&login_token={gigya_session_token}
&redirect_uri=dk.dsg.cpsag.netto://p
```

Android intercepts the redirect via `RedirectActivity` when the browser hits `dk.dsg.cpsag.netto://p?code=xxx`.

Step 4 — Auth code exchange

```
POST https://idp.dsgapps.dk/token
Content-Type: application/x-www-form-urlencoded

grant_type=authorization_code
&code={auth_code}
&code_verifier={pkce_verifier}
&client_id=customer-program
&redirect_uri=dk.dsg.cpsag.netto://p
```

Step 5 — Token refresh ✓ confirmed working

```
POST https://idp.dsgapps.dk/token
Content-Type: application/x-www-form-urlencoded

grant_type=refresh_token
&refresh_token={refresh_token}
&client_id=customer-program
```

Note: `client_id=customer-program` is required for refresh. `scan-and-go-native` returns `invalid_grant`.

Token response (exchange and refresh)

```
{
  "access_token": "eyJ...",
  "refresh_token": "7hsq...",
  "id_token": "eyJ..."
}
```

Logout / token revocation

```
POST https://idp.dsgapps.dk/token/revocation
Content-Type: application/x-www-form-urlencoded

token={token}
&client_id=customer-program
```

Token storage (SharedPreferences)

Confirmed keys in `dk.dsg.netto.xml` :

- `access_token` → used as `Authorization: Bearer` header
- `refresh_token` → used for silent refresh
- `token` (ID token) → used as `x-id_token: Bearer` header

Receipt API confirmed working ✓

```
requests.get("https://p-club.dsgapps.dk/api/cp/receipt", headers={
    "Authorization": f"Bearer {access_token}",
    "x-id_token": f"Bearer {id_token}",
})
# Returns full receipt history - HTTP 200
```

New account registration consent

First-time login requires accepting consents via Gigya. Required fields:

```
{
  "dk.salling.newsletter_clubsalling": {"isConsentGranted": true},
  "terms.dk.salling.clubsalling": {"isConsentGranted": true},
  "privacy.dk.salling.clubsalling": {"isConsentGranted": true},
  "profiling.dk.salling.clubsalling": {"isConsentGranted": true}
}
```

Submit via `accounts.setAccountInfo` with `regToken` + `finalizeRegistration=true`.

Heimdal API Endpoints

Base URL: `https://p-heimdalbackend.dsgapps.dk/api/`

Method	Endpoint	Description
GET	<code>store/supportedStores</code>	List all scan-and-go enabled stores
GET	<code>store/{id}</code>	Get store info
GET	<code>store/findByLocation?</code> <code>lat=&lng=&radius=</code>	Find nearby stores
GET	<code>user/me</code>	Get user/employee card
GET	<code>user/memberId</code>	Get member ID
GET	<code>receipts/{id}?</code> <code>sort=desc&page=&limit=</code>	Purchase history

Method	Endpoint	Description
GET	news?current=true	In-app news
GET	settings/native	App feature flags/config
GET	dev/store/terminal	Dev endpoint — all store terminals
POST	basket	Create/update basket (main scan endpoint)
PATCH	selectionalgorithm/{id}	Age verification decision
PATCH	user	Update employee card
PUT	user	Update user info

Auth Headers

Every request uses two headers injected by OkHttp interceptors:

```
Authorization: Bearer <user_access_token>    # OIDC access token
x-id-token:    Bearer <user_id_token>        # OIDC ID token
```

Token refresh is handled automatically on `AuthTokenExpiredError` (HTTP 400).

Club API Endpoints

Base URL: `https://p-club.dsgapps.dk/api/cp/`

Method	Endpoint	Description
GET	receipt	Full receipt list grouped by month
GET	receipt/details?type=merged&receiptId={id}	Full line-item detail for a specific receipt
GET	settings	App config and feature toggles

Receipt List — GET `/api/cp/receipt`

No parameters required. Returns all historical receipts for the authenticated user grouped by month, going back years.

Key fields per receipt entry:

Field	Description
-------	-------------

id	{storeId}-{posNumber}-{transactionNumber}-{postingDate}
postingDate	DSG internal epoch — do not use for date parsing
createdAt	Standard ISO timestamp — use this instead
salesTotal	Total in DKK (not øre)
memberDiscount	Membership discount in DKK
otherDiscount	Other discounts in DKK
storeName	Human-readable store name
type	merged = scan-and-go, full = traditional checkout

Response is grouped by month:

```
{
  "receiptsList": [
    {
      "groupTitle": "MARTS 2026",
      "groupSavingsTxt": "0,00 kr.",
      "receipts": [
        {
          "id": "7567-11-83-1774441701",
          "salesTotal": 65,
          "storeName": "Netto Eternitten",
          "type": "merged",
          "createdAt": "2026-03-27T10:32:13.211Z"
        }
      ]
    }
  ]
}
```

Receipt Detail — GET /api/cp/receipt/details? type=merged&receiptId={id}

Returns full line-item breakdown for a specific receipt.

Key fields per line item:

Field	Description
articleDescription	Product name
ean	EAN barcode
salesPrice	Price paid in DKK
normalPrice	Original price before discount
discount	Discount amount in DKK

discounts[]	Named discount list e.g. RABAT -11,00
qtyInSalesUnit	Quantity
taxAmount	VAT in DKK
itemType	01 = product, 02 = deposit (pant)
refundQualifier	Whether item can be refunded

```
{
  "lineItems": [
    {
      "articleDescription": "PEPSI MAX *",
      "ean": 5741000123676,
      "salesPrice": 9,
      "normalPrice": 20,
      "discount": 11,
      "discounts": [{"name": "RABAT", "value": "-11,00"}],
      "qtyInSalesUnit": 1,
      "taxAmount": 1.8,
      "itemType": "01"
    },
    {
      "articleDescription": "PANT",
      "ean": 8880173,
      "salesPrice": 3,
      "itemType": "02"
    }
  ],
  "address": {
    "brand": "Netto Eternitten",
    "street": "Alexander Foss Gade 30",
    "city": "9000 Aalborg"
  }
}
```

Polling Strategy

```
Poll GET /api/cp/receipt periodically
  → compare receipt IDs against last known list
  → new ID found → GET /api/cp/receipt/details?receiptId={id}
  → store in backend
```

Covers ALL purchases — scan-and-go and traditional checkout. No VPN or app modification required.

Key Data Models

ProductObjectDto — Single scanned item

Field	JSON key	Type	Description
barcode	ean	String	Scanned EAN barcode
vikingEan	—	String	Internal Viking EAN
name	description	String	Product name
quantity	quantity	Float	Supports weighted items
unitPrice	unitPrice	Long	Unit price in øre
totalPrice	totalPrice	Long	Total price in øre
weighted	isWeighted	Boolean	True for deli/produce
measuringUnit	measuringUnit	String	pcs , kg , etc.
related	related	List	Related items e.g. deposit (pant)

ProductBasketDto — Full basket

Field	Description
id	Basket UUID
storeId	Store UUID
items	List of ProductObjectDto
totalBasketPrice	Total in øre
subtotalBasketPrice	Subtotal in øre
tax / taxAfterDiscount	VAT in øre
memberId	Member ID
paidAt	Payment timestamp
tempReceiptBarCode	Receipt barcode before payment
membershipDiscountOffers	Available membership discounts

Note: All prices are stored in **øre** (1/100 DKK). Divide by 100 for DKK.

Basket API Behaviour

Key Finding — Everything is POST

All basket operations (add, remove one, remove all) use a single `POST /api/basket` with the **full basket state**. There is no DELETE or PATCH for basket items.

Action	Method	Result
Scan item	POST	Item appears in <code>items</code> array
Scan again	POST	Same item with quantity+1
Remove one	POST	Same item with quantity-1
Remove all	POST	Item absent from <code>items</code> array
Pay	POST	<code>paidAt</code> becomes non-null timestamp

Basket Lifecycle

```
POST /api/basket → paidAt: null      = basket in progress
POST /api/basket → paidAt: "2026-..." = purchase completed → persist as receipt
```

Diffing Strategy

Since the response always contains full basket state, track changes by diffing against previous state:

```
Previous: [Monster x3]
Current:  [Monster x1]
Diff:     Monster quantity decreased by 2
```

Request — `POST /api/basket`

Sent on every scan/update. Contains full basket state including all items.

Key fields:

```
{
  "id": "1f4f8638-0bff-4558-b6db-6e67e779aee6",
  "storeId": "00000000-0000-0000-0000-000000007558",
  "items": [
    {
      "ean": "5060337502290",
      "quantity": 2.0,
```

```
        "totalPrice": 0
      }
    ],
    "memberId": 3533071777,
    "paidAt": null
  }
}
```

Response

Server fills in all pricing data:

```
{
  "id": "1f4f8638-0bff-4558-b6db-6e67e779aee6",
  "closed": false,
  "paidAt": null,
  "items": [
    {
      "ean": "5060337502290",
      "description": "MONSTER ULTRAWHITE",
      "quantity": 2,
      "unitPrice": 1400,
      "totalPrice": 2800,
      "unitPriceWithoutDiscount": 1500,
      "totalVikingDiscount": 200,
      "vatPercent": 25,
      "measuringUnit": "pcs",
      "isWeighted": false,
      "related": [
        {
          "description": "PANT",
          "ean": "8880171",
          "unitPrice": 100,
          "totalPrice": 200
        }
      ]
    }
  ],
  "totalBasketPrice": 3000,
  "subtotalBasketPrice": 3000,
  "subtotalBasketPriceTaxExcluded": 2400,
  "taxAfterDiscount": 600
}
```

Frida Setup

Environment

- Conda env: `frida-env` (Python 3.11)
- Frida: 17.8.3
- Target: Android Emulator API 34 x86_64 (no Google APIs = root access)
- frida-server: `frida-server-17.8.3-android-x86_64`

GrapheneOS Note

Frida's Java bridge (`Java.perform`) does **not** work on GrapheneOS due to hardened ART. The emulator is required for development.

Start frida-server on emulator

```
adb root
adb shell "/data/local/tmp/frida-server/frida-server-17.8.3-android-x86_64"
```

Attach to app

```
frida -U -n "Netto+" -l hook_basket.js
```

Working Frida Hook

Intercepts basket updates and prints item name, quantity, unit price, total price and basket total.

```
Java.perform(function () {
    var RealCall = Java.use("okhttp3.internal.connection.RealCall");
    RealCall.execute.implementation = function () {
        var request = this.request();
        var url = request.url().toString();

        if (url.includes("heimdal") && url.includes("basket")) {
            var response = this.execute();
            var responseBody = response.peekBody(1000000);
            var json = JSON.parse(responseBody.string());

            console.log("\n=== BASKET UPDATE ===");
            json.items.forEach(function(item) {
                console.log(
```

```

        item.description +
        " x" + item.quantity +
        " @ " + (item.unitPrice / 100).toFixed(2) + " DKK" +
        " = " + (item.totalPrice / 100).toFixed(2) + " DKK"
    );
    });
    console.log("TOTAL: " + (json.totalBasketPrice / 100).toFixed(2) +
    " DKK");
    return response;
}

return this.execute();
};

console.log("Expense tracker running...");
});

```

Auth Strategy Decision

Decision: Option A — Independent auth session per app ✓ confirmed working

Our app performs its own independent login using the same Gigya + PKCE flow as the Netto app. Each app maintains its own separate session in DSG's IDP. No token conflicts — DSG's IDP issues separate independent token pairs per login session.

Why not share tokens from the Netto app:

- Refresh tokens are single-use and rotating
- Whichever app refreshes first invalidates the other's token
- Would cause `invalid_grant` errors in whichever app refreshes second
- Creates a fragile dependency on the Netto app being installed

Confirmed working auth parameters — `customer-program` client:

```

authorization endpoint: https://p-idp.dsgapps.dk/apps
clientId:               customer-program
channel:                CustomerProgram
clientFlow:             gigya
redirect_uri:           http://localhost   (registered server-side)
token endpoint:         https://idp.dsgapps.dk/token
token refresh client:   customer-program

```

Why customer-program and not scan-and-go-native :

- customer-program redirects to `http://localhost` — a completely different redirect URI
- scan-and-go-native redirects to `dk.dsg.cpsag.netto://p`
- Different redirect URIs = different OAuth clients = **100% independent sessions**
- Refreshing with customer-program will never invalidate the Netto app's scan-and-go-native tokens

Our app's auth flow (confirmed end-to-end):

1. Gigya `accounts.login` → session token
2. Gigya `accounts.getJWT` → Gigya JWT
3. Chrome Custom Tab opens `https://p-idp.dsgapps.dk/apps` with `login_hint=gigya_jwt`
4. User enters password → clicks Næste (email pre-filled from `emailOrPhone` param)
5. Browser redirects to `http://localhost/#id_token=...&code=...`
6. App catches redirect via `CustomTabsIntent` + `RedirectActivity`, extracts code from fragment
7. POST `https://idp.dsgapps.dk/token` with `authorization_code` + PKCE verifier → tokens
8. Tokens stored securely in Android Keystore
9. Silent refresh via POST `/token` with `refresh_token` + `client_id=customer-program`
10. User never logs in again

Receipt API confirmed working — 114 receipts across 15 months:

```
GET https://p-club.dsgapps.dk/api/cp/receipt
Authorization: Bearer {access_token}
x-id_token:    Bearer {id_token}
→ 200 OK
```

Next Steps

- ☐ Capture real basket POST with `paidAt` populated by completing a purchase on a non-GrapheneOS device
- ☐ Clarify `type: merged` vs `type: full` receipt distinction definitively
- ☐ Investigate `postingDate` epoch format
- ☐ Map payment flow endpoints on a real device with a saved card

- ☐ Handle weighted items (quantity is float, e.g. 0.453 kg) in basket hook
- ☐ Investigate dev/store/terminal endpoint
- ☐ Implement Chrome Custom Tab PKCE flow in Android app (mirrors the Selenium proof of concept)